Studio Nx

Backend(FastAPI/Python) 14 — check_password_rules()

2026-10-12

Backend(FastAPI/Python) 14 — `check_password_rules()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-04 회원가입: 비밀번호 규칙 (PRD 01 FR-102)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	13 모델 관계 (P1-03 끝) · 15 `hash_password()`

1. 하는 일

비밀번호가 규칙 6가지 중 **무엇을 어겼는지** 이름 목록으로 돌려줍니다. 빈 목록이면 통과입니다.

`check_password_rules("abcdefgh")` → `["upper", "number", "special"]`

이름	규칙	join.html 조건 목록의 해당 줄
`length`	8 ~ 20자	8–20 characters
`upper`	대문자 1개 이상	Uppercase letter
`lower`	소문자 1개 이상	Lowercase letter
`number`	숫자 1개 이상	Number
`special`	키보드 기호 1개 이상 (`!@#$%^&*` 등)	Special character (!@#$...)
`allowed`	영문, 숫자, 키보드 기호만 허용 (한글과 공백은 불가)	(목록에는 없고, 칸 아래 오류 문구로 표시합니다)

2. 언제 쓰나

쓰는 곳	하는 일
P1-04 `signup()` (08)	가입 요청의 비밀번호를 검사합니다. 실패하면 422로 응답합니다
P1-09 `change_password()`	새 비밀번호를 같은 규칙으로 검사합니다

브라우저도 같은 규칙으로 검사합니다 (`join.js`의 `PASSWORD_RULES`, P1-07). 입력할 때마다 조건 목록에 ✓를 붙입니다. **그런데도 서버가 한 번 더 검사하는 이유**는, 브라우저 검사는 개발자 도구나 직접 만든 요청으로 건너뛸 수 있기 때문입니다.

검사에 실패하면 `signup()`이 이 목록을 그대로 담아 응답합니다.

```
422 {"error": {"code": "VALIDATION_ERROR", "message": "Check the password requirements.",
               "details": {"password": ["upper", "number", "special"]}}}
```

화면은 `details.password`에 담긴 이름으로 join.html 조건 목록의 해당 줄(`li[data-rule="upper"]` 등)을 표시할 수 있습니다.

3. 코드

import (파일 맨 위)

```
import re
import string

import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다. **`re`와 `string` 두 줄을 이번에 추가했고**, 이 함수는 그 둘만 사용합니다. 둘 다 파이썬 기본 모듈이라 설치할 필요가 없습니다.

import	가져오는 것	쓰는 곳
`re`	정규식 (글자 패턴 찾기)	대문자, 소문자, 숫자, 허용 문자 검사
`string`	자주 쓰는 글자 모음	`string.punctuation`: 키보드 기호 32개

함수

```
PASSWORD_MIN, PASSWORD_MAX = 8, 20


def check_password_rules(plain: str) -> list[str]:
    failed: list[str] = []
    if not PASSWORD_MIN <= len(plain) <= PASSWORD_MAX:
        failed.append("length")
    if not re.search(r"[A-Z]", plain):
        failed.append("upper")
    if not re.search(r"[a-z]", plain):
        failed.append("lower")
    if not re.search(r"[0-9]", plain):
        failed.append("number")
    if not any(ch in string.punctuation for ch in plain):
        failed.append("special")
    if not re.fullmatch(r"[!-~]+", plain):
        failed.append("allowed")
    return failed
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와서 `모듈.이름`으로 씁니다 (예: `re.search`, `string.punctuation`)

식	뜻
`PASSWORD_MIN, PASSWORD_MAX = 8, 20`	두 변수에 한 번에 값을 넣습니다 (`PASSWORD_MIN = 8`, `PASSWORD_MAX = 20`)
`8 <= len(plain) <= 20`	파이썬은 비교를 이어서 쓸 수 있습니다. `8 <= len(plain) and len(plain) <= 20`과 같습니다
`r"[A-Z]"`	정규식입니다. `[A-Z]`는 A부터 Z 중 한 글자를 뜻합니다. 앞의 `r`은 `\`를 글자 그대로 쓰겠다는 표시입니다
`re.search(패턴, 글)`	글 **어딘가에** 패턴이 있으면 결과를, 없으면 `None`을 돌려줍니다. `not None`은 `True`입니다
`re.fullmatch(패턴, 글)`	글 **전체가** 패턴과 맞아야 결과를 돌려줍니다

식	뜻
`[!-~]+`	`!`(33번)부터 `~`(126번)까지의 글자가 1개 이상이라는 뜻입니다. 영문, 숫자, 키보드 기호가 모두 이 범위 안에 있고, 한글과 공백(32번)은 범위 밖입니다
`any(ch in string.punctuation for ch in plain)`	글자를 하나씩 보면서 기호인 것이 **하나라도** 있으면 `True`입니다

입력	결과
`"Abcdef1!"`	`[]` (통과)
`"Ab1!xyz"` (7자)	`["length"]`
`"abcdefgh"`	`["upper", "number", "special"]`
`"Abcdef1!한"`	`["allowed"]`
`"Abc def1!"` (공백)	`["allowed"]`
`""`	6개 모두

4. 흐름

1. 빈 목록 `failed`를 만듭니다.
2. 규칙 6개를 **차례대로 모두** 검사하고, 어긴 규칙의 이름을 넣습니다.
3. 목록을 돌려줍니다.

5. 왜 이렇게

선택	이유
처음 실패한 규칙에서 멈추지 않고 모두 검사	화면이 어긴 조건을 한 번에 모두 표시할 수 있습니다
`True`/`False`가 아니라 이름 목록을 돌려줌	화면의 조건 줄 이름(`data-rule`)과 같아서 그대로 연결됩니다
이 함수는 오류를 던지지 않음	가입(08)과 비밀번호 변경(P1-09)이 같은 함수를 쓰고, 오류 응답은 각자 만듭니다
기호는 `string.punctuation`으로 판단	파이썬이 정해 둔 키보드 기호 32개입니다(`!"#$%&'()*+,-./:;<=>?@[\]^_{|}~`). `join.js`의 `/[!-/:-@[-{-~]/`와 같은 모음인지 시험으로 확인했습니다
최대 20자	PRD FR-102에 정해진 값입니다. bcrypt(15)는 72바이트까지만 쓰므로, 20자는 그보다 충분히 짧습니다

6. 시험 (`tests/test_password.py` 새 파일, 16개)

시험	확인한 것
통과 4개	8자, 20자, 같은 글자 반복, 밑줄 기호는 모두 `[]`입니다
실패 10개	7자, 21자, 대문자 없음, 소문자 없음, 숫자 없음, 기호 없음, 한글, 공백, 여러 규칙 동시 위반, 빈 문자열은 위 표의 목록과 같습니다
모든 기호	`string.punctuation` 32개가 각각 `special`로 인정됩니다
화면과 같은 규칙	ASCII 33~126에서 영문과 숫자를 뺀 모음이 `string.punctuation`(= `join.js` 정규식)과 같습니다

전체 시험 92개가 통과했고, `ruff` 검사도 통과했습니다.